Session 7 · Deck 7.B3

# Architecture as a Constraint on $W$: Locality, and Shape You Can Guarantee

Zhigang Feng

Thursday, November 5, 2026

© Zhigang Feng • Redistribution prohibited without permission

*Build it on paper → solve it on the machine → push it to the frontier with AI*

# Where We Are

you are here: 7.B3 ▼

| S1 ✓ | S2 ✓ | S3 ✓ | S4 ✓ | S5 ✓ | S6 ✓ | S7 | S8 | S9 | S10 |
|---|---|---|---|---|---|---|---|---|---|
| Sep 24 | Oct 1 | Oct 8 | Oct 15 | Oct 22 | Oct 29 | Nov 5 | Nov 12 | Nov 19 | Dec 3 |
| Intro | HA · OLG | Num. DP | Numerics | Numerics | HA classic | ML | RL | Text | Agents |
| Growth | Policy | Python | I | II | The wall | Deep nets | HA + DL | LLMs | Projects |
| trunk | branches | the machine · classical methods | | | the wall | new tools | | new data | colleagues |

*Build it on paper → solve it on the machine → push it to the frontier with AI*

**Block A — learning a function:** 7.A1 the learning problem and the neuron · 7.A2 activations and approximation · 7.A3 backpropagation and training. **Block B — using it:** 7.B1 PyTorch and a forecast · 7.B2 the growth model · **7.B3** architecture and shape · 7.B4 capacity and order.

© Zhiaki Feng · Redistribution prohibited without permission

# Roadmap: Three Parts

**Restrictions on $W$ that encode what we know: about the input, and about the target.**

| Part | Topic | The question it answers |
|---|---|---|
| 1 | Architecture as a constraint on $W$ | What does restricting a layer claim about the input, and what do convolution, recurrence and graphs have in common? |
| 2 | Locality: the convolution | What does a dense layer ignore about an image, and what do two constraints on $W$ — sparse, and repeated — buy? |
| 3 | Shape you can guarantee | How do you make a network monotone or convex by construction, and what does the guarantee cost? |

Redistribution prohibited without permission
Yiqiang Feng

**Idea**

Every layer here is the same $\sigma(Wx + b)$. They differ only in which entries of $W$ are free, fixed at zero, forced equal or forced non-negative — and each choice is a claim, about the input or about the target, that can be wrong.

# The Same Layer, with Restrictions

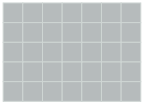
**Every restriction on $W$ is a claim about the target.**

- Every network so far has been **dense**: every entry of every $W$ free, nothing assumed about the input.
- A dense layer is indifferent to its input's structure. Shuffle the pixels of every digit the same way and it learns exactly as well — which means it never used the fact that pixels have neighbours.
- **An architecture is a constraint on $W$**: entries fixed at zero, entries forced equal, entries forced non-negative. Each one is a claim about the target.
- Three of those claims are worth an economist's time: locality (convolutions), shape (monotone and convex policies), and order — or its absence (sequences, networks of agents, 7.B4).
- **Then the opposite question: what does an *un*restricted network buy from having more parameters than observations? (7.B4)**

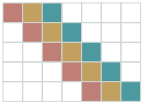
© Zhiyang Feng · Redistribution prohibited without permission

# One Idea, Three Times

**A constraint on $W$ is a prior about the target. True, it saves parameters and data. False, it is bias.**



**dense**
every entry free



**convolution**
sparse and repeated



**recurrent**
one block at every date



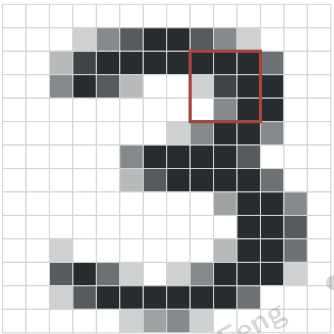
**graph**
follows the links

© Lirang Feng · Redistribution prohibited without permission

- A layer is $\sigma(Wx + b)$. White cells are fixed at zero; in the dense matrix every grey cell is free; in the other three, cells of one colour are *one* shared parameter.
- A dense $W$ assumes nothing about the input, so everything must be learned from data.
- **A constraint on $W$ is a prior about the target. True, it saves parameters and data. False, it is bias.**

Miao & Feng, *AI for Economists*, §9.6.

**Two facts about images are unused: nearby pixels belong together, and a pattern can appear anywhere.**

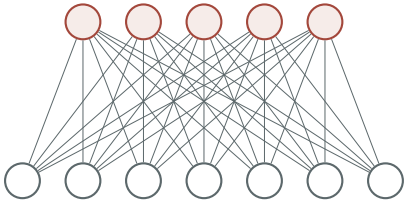


- 7.A1 flattened the image into a vector. A dense first layer then has one weight per pixel per hidden unit: $196 \times 128 = 25{,}088$ weights for the $14 \times 14$ digit here, $784 \times 128 = 100{,}352$ for MNIST's $28 \times 28$.
- Shuffle the pixels, the same way in every image, and a dense network learns exactly as well. It never knew which pixels were neighbours.
- Yet a stroke is a local object: the red $3 \times 3$ patch means the same thing wherever it sits.
- **Two facts about images are unused: nearby pixels belong together, and a pattern can appear anywhere.**

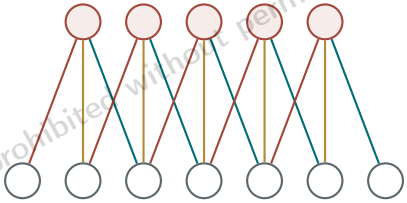
© Zhigang Feng
Redistribution prohibited without permission

# Dense vs Convolution: 35 Weights vs 3

**Convolution is a dense layer after two constraints on $W$: sparse, and repeated.**



**dense**: 35 lines, 35 free weights



**convolution**: 15 lines, 3 free weights

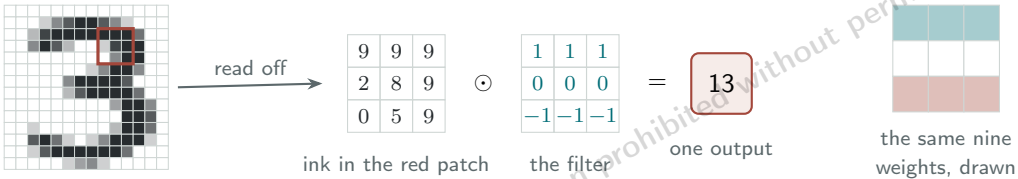
© Zhiyang Feng Redistribution prohibited without permission

- Seven inputs, five outputs. On the right each output sees three adjacent inputs (**local connection**), and lines of one colour are one parameter (**weight sharing**).
- The three shared weights are the **filter**, or **kernel**. Sliding it along the input is a **convolution**.
- **Convolution is a dense layer after two constraints on $W$: sparse, and repeated.**

Miao & Feng, *AI for Economists*, §9.6, Fig. 9.9.

# The Receptive Field

**One output sees one $3 \times 3$ patch: an inner product with nine weights.**



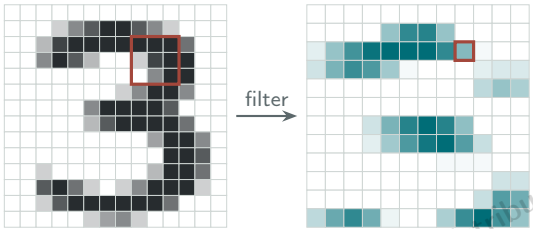

© Zuheir Ismail · Redistribution prohibited without permission

- The **receptive field** of an output is the part of the input it can see: here the $3 \times 3$ patch outlined in red, and nothing else.
- The output is an inner product: $(9 + 9 + 9) - (0 + 5 + 9) = 13$. The filter asks whether there is ink above and paper below, so it fires on the lower edge of a horizontal stroke.
- **Drawn as a picture it is 7.A1's template neuron, shrunk to nine weights — and those nine are reused at every position on the next frame.**

**Patch: rows 3 to 5, columns 9 to 11 of the $14 \times 14$ digit of 7.A1.**

# Feature Maps: One Question Asked Everywhere

**One filter is one question, asked at every location with the same nine weights.**

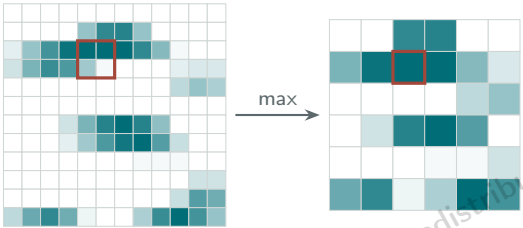



- Slide the filter over every position, apply a ReLU, and record the answers: a **feature map**.
- It lights up along the lower edges of horizontal strokes, wherever they are; its largest value is 27.
- A layer has many filters, often 32 or 64, each with its own map. They are learned, not designed.
- **One filter is one question, asked at every location with the same nine weights.**

Computed from the digit of 7.A1 with the filter of the previous frame; the red cell is the value 13.

© Zhigang Feng
Redistribution prohibited without permission

**Convolution extracts features; pooling reduces resolution, trading location for robustness.**




- **Pooling** replaces each small block by one summary number. **Max pooling** keeps the largest.
- The red $2 \times 2$ block holds $27, 27, 10, 1$; it becomes $27$. Resolution halves: $12 \times 12 \to 6 \times 6$.
- The question becomes "is the edge somewhere in this block?" Pooling has no parameters.
- **Convolution extracts features; pooling reduces resolution, trading location for robustness.**

© Zhigang Feng · Redistribution prohibited without permission

# The Full Pipeline: Equivariance, then Approximate Invariance

**A convolutional network is a learned feature extractor followed by the network you already know.**

A colour image now, not the digit: three channels, $32 \times 32$ pixels.




© Zheng Feng — Redistribution prohibited without permission

- **Equivariance**: shift the input one pixel and the feature map shifts one pixel (weight sharing).
- **Invariance**: the output does not change. Pooling gives it approximately, for small shifts.
- Of 281,665 parameters the convolutions hold 6.9%; the dense head holds 93.1%.
- **A convolutional network is a learned feature extractor followed by the network you already know.**

Miao & Feng, *AI for Economists*, §9.6, Fig. 9.10.

# What the Constraint Buys, and What It Assumes

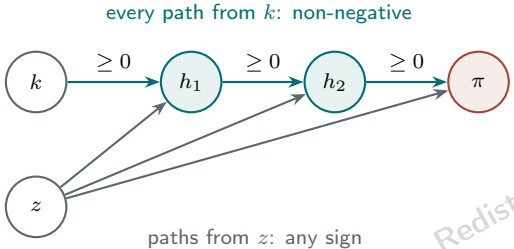
**Parameters saved are paid for with an assumption. Check the assumption, not the parameter count.**

- A $256 \times 256$ colour image is $196{,}608$ numbers: a dense layer into 256 units needs about $5 \times 10^7$ weights, while sixty-four $3 \times 3$ filters over three colour channels need $64 \times (3 \cdot 3 \cdot 3 + 1) = 1{,}792$ — four orders of magnitude fewer.
- Fewer parameters need fewer observations. In economics the sample is what it is, so this is decisive.
- The assumption bought: nearby inputs are related, distant ones are not, and a pattern means the same thing everywhere.
- Apply a convolution to data with no spatial order, such as the columns of a survey, and the constraint is simply wrong.
- Random, untrained filters with a ridge regression on top match deep training on satellite tasks (Rolf et al., 2021): the locality and the sharing do the work.
- **Parameters saved are paid for with an assumption. Check the assumption, not the parameter count.**

© Zhirang Feng · Redistribution prohibited without permission

E. Rolf et al., *Nature Communications* 12, 2021; Miao & Feng, *AI for Economists*, §9.6.

**Non-negative weights on every path from $x_j$, and non-decreasing activations, give $\partial f/\partial x_j \geq 0$.**



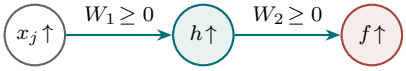

- Theory often gives the shape: saving rises in wealth, demand falls in own price.
- A **monotone network**: every weight on every path from $x_j$ non-negative and activations non-decreasing give $\partial f/\partial x_j \geq 0$ everywhere.
- *Partial* monotonicity is the useful case: constrain the paths from $k$, leave those from $z$.
- **Impose $W \geq 0$ by training $V$ in $W = \mathrm{softplus}(V)$; a clip stalls the gradient at the boundary.**

© Zhigang Feng
Redistribution prohibited without permission

J. Sill, "Monotonic networks", *NeurIPS* 10, 1998.

# 3 · Monotone Networks: Why the Sign Survives, and Where Theory Supplies It

**Each layer passes "larger in, larger out" to the next.**



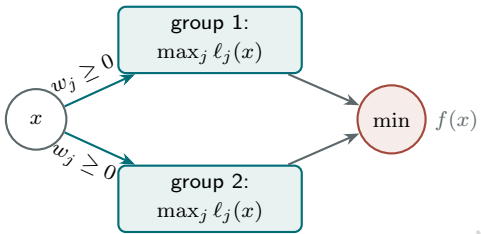

each layer: a larger input gives a larger pre-activation, and a non-decreasing $\sigma$ passes it on

- **Both conditions are needed.** One negative weight on a path from $x_j$ breaks it; so does a non-monotone activation (GELU and Swish dip below zero).
- **Decreasing instead**: feed $-x_j$, e.g. minus own price for a demand curve.
- **Where theory gives the sign**: saving $a'(a, z)$ rising in assets (6.A1's policy); consumption rising in cash on hand; demand falling in own price; a value function rising in the resource state.
- **What it prevents**: the spurious wiggles of an unconstrained fit, which a residual plot can hide.
- **Clip or reparameterise?** A weight clipped at zero gets no gradient and stays dead; $W = \mathrm{softplus}(V)$ does not.

© Zhiang Fan — redistribution prohibited without permission

J. Sill, *NeurIPS* 10, 1998; Miao & Feng, *AI for Economists*, ch. 9.

**A harder search, not a smaller class.**




$\ell_j(x) = w_j x + b_j$:
Sill's min–max network

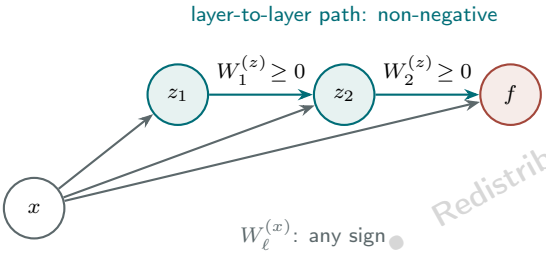
- Sign constraints shrink the parameter set, so a constrained network often trains more slowly and to a *higher* training loss.
- That is a harder search, not a smaller class: maxima within groups of linear units, a minimum across groups, and the family is universal among monotone functions.
- Penalising violations instead stays flexible, and holds only where it was sampled.
- **The unconstrained network reaches its lower loss partly with functions the theory forbids.**

© Zhigang Feng
Redistribution prohibited without permission

J. Sill, "Monotonic networks", *NeurIPS* 10, 1998, §2 (the min–max architecture).

# 3 · Input-Convex Networks: Which Weights, Which Arguments, Which Direction

**Constrain the layer-to-layer weights; leave the direct weights from $x$ free.**




- An **input-convex network**: $z_{\ell+1} = \sigma\left(W_\ell^{(z)} z_\ell + W_\ell^{(x)} x + b_\ell\right)$.
- Convex in $x$ if $W_\ell^{(z)} \geq 0$ and $\sigma$ is convex and non-decreasing (ReLU, softplus). The direct weights $W_\ell^{(x)}$ are free.
- Concave: take $-f$. Convex in some arguments only: constrain only their paths.
- **Examples: a cost function convex in input prices; a production function concave in capital.**

© Zhigang Feng
Redistribution prohibited without permission

B. Amos, L. Xu, J. Z. Kolter, "Input convex neural networks", *ICML*, 2017.

# Why the Network Is Convex: Three Facts and an Induction

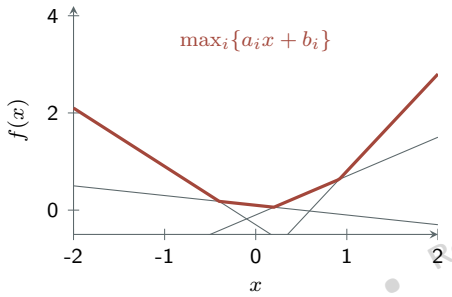
**Every layer, and the output, is convex in $x$ at every value of the parameters.**

- **Base.** $z_0 = x$ is linear in $x$, hence convex.
- **Step.** Suppose every unit of $z_\ell$ is convex in $x$. Then
  - $W_\ell^{(z)} z_\ell$ is a *non-negative* combination of convex functions: convex;
  - adding the affine term $W_\ell^{(x)} x + b_\ell$, of any sign, keeps it convex;
  - a convex, non-decreasing $\sigma$ of a convex function is convex.
- So every layer, and the output $f$, is convex in $x$, at every value of the parameters.
- **Both conditions carry weight.** A negative $W^{(z)}$ flips a convex unit to concave; a non-monotone $\sigma$ such as $\tanh$ breaks the last step.
- **The free direct weights are why it can still fit**: each layer sees $x$ again and can tilt and place the function.

© Zhigang Feng · Redistribution prohibited without permission

B. Amos, L. Xu, J. Z. Kolter, "Input convex neural networks", *ICML*, 2017, Prop. 1.

**The economist's usual position: theory pins the shape, not the curvature.**




- **Theorem**: an input-convex network approximates any convex function on a compact domain.
- Why: a maximum of affine functions is convex (left), every convex shape is a limit of such maxima, and a ReLU input-convex network is a nested max-affine map.
- No CES, Cobb–Douglas or translog: it imposes the *shape*; the data choose within it.
- **The economist's usual position: theory pins the shape, not the curvature.**

© Zhigang Feng redistribution prohibited without permission

Y. Chen, Y. Shi, B. Zhang, "Optimal control via neural networks: a convex approach", *ICLR*, 2019. Figure: schematic.

**Impose the restriction exactly where theory gives one, and nowhere else.**

- Point the same input-convex network at $\sin 3x$ on $[-1, 1]$: median RMSE $0.345$, and the fit stays convex to within $9 \times 10^{-11}$. It cannot bend, which is the guarantee seen from the other side.
- **A structured network that cannot fit is saying the imposed theory is wrong**: an informative failure.
- A value function $V(k, z)$ is concave in the endogenous state $k$ and of no particular shape in the shock $z$. Impose convexity on $z$ too and you add bias that no residual reveals.
- **Partially input-convex** network: $z_{\ell+1} = \sigma\big(W_\ell^{(z)} z_\ell + W_\ell^{(u)}(u \circ v) + W_\ell^{(v)} v + b_\ell\big)$, $W_\ell^{(z)} \geq 0$; capital is the constrained input $u$, the shock the free input $v$.
- Concave $V$ makes $\max_{k'}\{u(c) + \beta V(k')\}$ one basin, so gradient steps find its maximum (Session 8).
- **Impose the restriction exactly where theory gives one, and nowhere else.**

© Zhigang Feng · Redistribution prohibited without permission

Amos, Xu & Kolter, *ICML*, 2017, §3.2. $\sin 3x$: this course's earlier computations (2026), not recomputed in Lab 7.

# Measured: What an Unconstrained Network Does to Convexity

**The constrained network buys a guarantee and pays for it in fit.**

| | median RMSE | worst negative curvature |
|---|---|---|
| plain ReLU network | 0.00172 | 5.93 (range 0 to 10.4 over seeds) |
| input-convex network | 0.01345 | below $7.1 \times 10^{-11}$ |

- Target: $0.6x^2 + 0.25x + 0.1$, exactly convex, curvature 1.2 everywhere. 1,153 and 1,186 parameters, five seeds.
- The plain network fits the *level* eight times better and is locally concave by about 5.9. Its loss shows none of that.
- At least one seed in five came out convex anyway: you cannot tell without checking.
- **The constrained network buys a guarantee and pays for it in fit: 7.8 times the RMSE.**

© Zhiyang Feng · Redistribution prohibited without permission

Recorded in this course's earlier computations (2026); not recomputed in Lab 7.

**Reliability, well-behaved downstream problems, data efficiency — at the price of flexibility.**

- **Buys reliability.** The shape holds at every $\theta$, every seed, and outside the training band, where data cannot enforce it.
- **Buys well-behaved downstream problems.** Convex costs give monotone factor demands, so a market-clearing price is unique and a calibration loop converges — 6.A2's bisection needed exactly that sign.
- **Buys data efficiency.** Fewer functions to choose between means fewer states to pin one down.
- **Refuses flexibility.** If the true function violates the shape, the constraint is bias. It must come from theory, not convenience.
- Session 8 uses the same device for value functions.
- **7.B2 imposed exactly one of these, feasibility, because it is the one theory guarantees on every rung.**

© Thiago Fernandes · Redistribution prohibited without permission

# Takeaways

- **Part 1.** An architecture is a prior about the input: entries of $W$ fixed at zero, forced equal, or forced non-negative. True, it saves parameters and data. False, it is bias.
- **Part 2.** A convolution is a dense layer after two constraints: local, and the same filter everywhere. On 7.A1's digit one filter is one question asked everywhere; pooling trades location for robustness; 1,792 weights do what a dense layer needs $5 \times 10^7$ for — if the input really is spatial.
- **Part 3.** Sign constraints on $W$ buy a guarantee and pay for it in fit: 7.8 times the RMSE on the convexity test, and a harder search — not a smaller class. Shape is the constraint an economist can usually justify, because theory delivers monotonicity and convexity before any data do.

© Zhigang Feng · Redistribution prohibited without permission

# Bridge: What More Parameters Buy

Every network in this deck was restricted: entries of $W$ set to zero, tied together or kept non-negative, each because something about the input or the target was known.

7.B4 makes the opposite move: no constraint at all, and more parameters than observations. Which of the many exact fits does training pick, and why does it generalise? Then the third claim about the input: order — or its absence.

**Next: 7.B4**

Double descent and implicit regularisation: from zero, gradient descent picks the smallest exact fit. Then recurrent networks, whose hidden state is a state variable, and graph networks, whose permutation invariance is the restriction heterogeneous-agent models impose (Session 8).

© Zhigang Feng Redistribution prohibited without permission